

阶段7：多智能体协作优化实施方案

阶段7：多智能体协作优化实施方案

状态：Proposed

更新日期：2026-08-06

参考资料：Claude Code 源码：Multi-Agent 机制.pdf

适用范围：Question Evolution 智能体控制系统后续优化

目标读者：后续实现人员、人工复核人员、实验负责人

读完后应能做的事：明确本项目哪些环节适合多智能体、每个智能体能做什么、不能做什么、如何并行、如何审计、如何测试。

1. 结论

本项目后续可以引入多智能体，但不能把它做成多个智能体共同执行实验。更合适的形式是：**一个主控负责执行和决策，多个专项智能体只读证据并给建议。**

借鉴 Claude Code 的做法后，本项目需要补齐 6 个工程机制：任务注册表、工具权限隔离、上下文隔离、异步任务记录、并行调度、结果合成与降级。补齐这些机制后，多智能体可以用于实验复盘、记忆编译、候选计划生成和人工复核预审。

一句话边界：**专项智能体可以分析和建议，不能直接改正式实验结果、正式记忆、正式提示词、正式算子和正式评分。**

2. 不直接照搬的内容

Claude Code 中有三类多智能体机制：常规 Subagent、Fork Subagent、Coordinator。放到本项目中应这样取舍：

本项目的目标不是“智能体越多越好”，而是让不同分析职责分开运行，并且每条建议都能追溯、能拒绝、能人工复核。

3. 前置条件

实施本方案前，项目至少应具备以下能力：

1. 已有 `AgentTask`、`AgentPlan`、`agent_run_state.json` 和 `agent_events.jsonl`。
2. 工具调用已经通过工具注册表执行，不能让智能体任意执行命令。
3. 观察器能读取实验目录并输出结构化观察。
4. 报告器能输出 `agent_report.md`。
5. 策略校验能拒绝越权行为，例如修改正式提示词、发布激活态全局记忆、绕过评分或跳过 manifest 校验。

如果这些能力没有完成，不应先做多智能体。否则会出现多个智能体同时给建议，但没有统一状态、统一证据和统一拒绝规则。

4. 总体结构

多智能体协作采用“主控 + 协调器 + 专项智能体”的结构。

代码块

- 1 用户任务
- 2 -> 主控智能体生成计划
- 3 -> 确定性执行器调用现有工具
- 4 -> 观察器生成结构化观察
- 5 -> 证据包生成器生成 `evidence_pack.json`
- 6 -> 协调器选择需要运行的专项智能体
- 7 -> 专项智能体并行读取证据包并输出 `advice`
- 8 -> 汇总器合并 `advice`，拒绝越权建议
- 9 -> 策略校验器检查可执行动作
- 10 -> 主控智能体决定报告、续跑、复盘或请求人工复核

主控智能体只有一个。它负责最终计划、工具调用授权、停止判断和报告输出。

协调器只负责分发专项任务、收集结果、合并建议，不直接执行实验工具。

专项智能体可以有多个。它们只读证据包，输出结构化建议，不直接写正式实验目录。

5. 任务注册表

每个专项智能体必须先登记到 `advisor_registry.py`，没有登记的智能体不能运行。

建议定义 `AdvisorSpec`：

代码块

- ```
1 {"advisor_id": "router_diagnosis", "name": "路由诊断智能体", "stage":
 "post_experiment_review", "purpose": "判断失败是否来自算子选择不合适", "trigger":
```

```
"experiment_finished", "allowed_inputs": ["evidence_pack.summary",
"evidence_pack.observations", "evidence_pack.evidence_refs"], "allowed_tools":
["read_evidence_pack", "read_artifact_ref", "write_temp_advice"], "model_tier":
"reasoning_high", "fallback_model_tier":
"reasoning_medium", "requires_json_output": true, "requires_evidence_refs":
true, "output_schema": "advisor_advice.schema.json", "max_runtime_seconds":
120, "max_input_chars": 30000, "retry_count": 1, "failure_policy":
"warn_and_continue"}}
```

注册表必须说明 6 件事：这个智能体什么时候运行、能读什么、能用什么工具、使用哪个模型能力层级、输出什么结构、失败后怎么处理。

注册表不能直接写死具体模型名称，例如“某个诊断智能体永远使用 GPT”或“某个搜索智能体永远使用 Qwen”。具体模型应由 `model_router.py` 根据 `model_tier` 和当前可用模型配置选择，这样后续替换模型时不需要改每个专项智能体。

## 6. 工具权限隔离

借鉴 Claude Code 的工具隔离思路，本项目对专项智能体采用“全局禁止 + 按类型白名单”。

### 6.1 全局禁止

所有专项智能体都不能使用以下能力：

- 1. 不能启动其它专项智能体，避免递归调用。
- 2. 不能向用户提问，用户只和主控智能体交互。
- 3. 不能调用 `run_loop.sh`、`resume_run_loop.sh` 或其它真实实验执行工具。
- 4. 不能修改 `prompts/`、`operator_router.py`、`llm_operator_router.py`、`scoring.py`、`prompts/operators/`。
- 5. 不能写入激活态全局记忆。
- 6. 不能修改正式实验目录中的评分、状态、manifest 和 branch 结果。
- 7. 不能把自动分析写成“已确认有效边界”。

### 6.2 按类型白名单

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

白名单之外的工具默认拒绝。拒绝结果必须写入 `advisor_events.jsonl`，不能静默忽略。

## 7. 上下文隔离

Claude Code 的关键经验是：上下文不能简单共享，也不能完全清空，而要按照字段决定。本项目建议按下面方式处理。

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

专项智能体不能直接扫描完整实验目录。主控流程先生成证据包，再按 `AdvisorSpec.allowed_inputs` 裁剪出该智能体需要的输入。

例子：评分稳定性智能体只需要评分规则摘要、重复评分记录、效果分析和分支结果引用，不需要完整题目生成日志和完整模型回复。

为便于和阶段 9 的上下文缓存方案对齐，每次专项智能体调用都应拆成 4 个上下文块：

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

专项智能体只能读取 `evidence_pack_slice`，不能继承完整父上下文。报告改写和同一证据包追加分析可以 Continue；专项诊断、冲突审查和验证类任务必须新建上下文，不能复用生成类智能体的历史判断。

## 8. 任务状态与审计

每次专项智能体运行都必须生成一条任务记录，建议命名为 `AdvisorRunRecord`。

代码块

```
1 {"advisor_task_id": "adv_20260806_001_router","parent_run_id": "agent-20260806-001","advisor_id": "router_diagnosis","status": "completed","model_tier": "reasoning_high","selected_model": "configured-reasoning-model","fallback_used": false,"input_hash": "sha256:...", "output_hash": "sha256:...", "started_at": "2026-08-06T10:00:00Z", "ended_at": "2026-08-
```

```
06T10:01:12Z","evidence_refs":["round_1/branch_results.jsonl#branch-001"],"error_summary":""}
```

状态只允许以下值：`pending`、`running`、`completed`、`failed`、`skipped`、`timeout`、`rejected_by_policy`。

新增事件文件 `advisor_events.jsonl`，至少记录：

1. `advisor_started`：专项智能体开始运行。
2. `advisor_completed`：专项智能体正常完成。
3. `advisor_failed`：专项智能体报错。
4. `advisor_timeout`：专项智能体超时。
5. `advisor_policy_rejected`：请求了禁止工具或越权动作。
6. `advice_merged`：建议被合并到报告或候选池。

这些事件只追加，不覆盖。后续复盘时可以直接从事件链还原“哪个智能体在什么证据上给了什么建议”。

## 9. 通信方式

本项目不需要先实现复杂消息队列，但要采用“文件化消息”的形式，避免专项智能体直接互相调用。

代码块

- 1 主控/协调器
- 2 -> 写入 `advisor_input.json`
- 3 -> 启动专项智能体
- 4 -> 专项智能体写入 `advisor_output.json`
- 5 -> 协调器读取输出并写入 `advice_merge.json`

后续如果需要支持长任务，可以增加 `advisor_messages.jsonl`：

代码块

- 1 `{"to":"router_diagnosis","message":"只检查 016 和 018 的 not_applicable 分布","created_at":"..."}`

第一版不允许专项智能体之间互发消息。所有追加指令都必须由协调器写入，防止多个智能体形成不可控对话。

## 10. 并行调度

独立专项智能体应并行运行，存在依赖的智能体必须串行运行。

## 10.1 可以并行的任务

实验结束后，以下智能体可以并行：

1. 路由诊断智能体。
2. 算子生成诊断智能体。
3. 校验诊断智能体。
4. 评分稳定性智能体。
5. 搜索成本智能体。

人工复核预审中，边界质量、可回答性、泄漏风险、机制命中也可以并行；复核汇总智能体必须等它们完成后再运行。

## 10.2 必须串行的任务

记忆编译必须按下面顺序：

代码块

- 1 事实抽取 + 分类映射
- 2 -> 策略归纳
- 3 -> 冲突审查
- 4 -> 发布预审

原因是策略归纳依赖事实和分类，发布预审依赖冲突审查结果。

## 10.3 并发限制

默认同时最多运行 4 个专项智能体。每个智能体默认超时 120 秒，超过后标记为 `timeout`，不阻塞主报告生成。

如果是大批量记忆编译，可把并发数降到 2，避免多个智能体同时读取大型日志导致成本和耗时失控。

## 10.4 模型路由策略

派生子智能体时需要按任务类型选择模型，但不能让子智能体自己选模型。子智能体只声明任务能力要求，协调器通过 `model_router.py` 选择具体模型。

建议采用下面的路由方式：

代码块

- 1 专项智能体类型
- 2 -> 任务能力要求
- 3 -> 模型能力层级

模型能力层级建议先定义为 4 类：

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

具体任务建议如下：

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

必须遵守 5 条规则：

- 1. 子智能体不能自己选择模型，只能声明 `model_tier`。
- 2. `model_router.py` 根据 `model_tier`、可用模型、预算和任务风险选择具体模型。
- 3. 搜索和摘录类智能体可以使用低成本模型，但输出只能是证据、匹配模式和短摘要。
- 4. 策略归纳、冲突审查、评分稳定性和建议合成不能只靠低成本模型完成。
- 5. 生成类任务和验证类任务不能复用同一个子智能体上下文；条件允许时应使用不同模型或至少新建上下文。

例子：

代码块

- ```
1  搜索智能体使用 Qwen3.6-27B 摘录 016 的 not_applicable 证据。
2  路由诊断智能体使用强推理模型判断这些证据是否说明 016 适用条件过宽。
3  建议合成智能体再判断该建议能否进入 needs_human_review 或 proposal。
```

这样可以把“便宜模型做证据整理，强模型做判断和合成”固定成工程规则，而不是写成某个提示词里的临时建议。

11. 触发规则

不要每轮都运行全部专项智能体。协调器应根据观察结果选择需要运行的智能体。

触发规则必须写成确定性代码，不能只写在提示词里。提示词只解释任务，不能决定是否越权运行。

12. Continue 与 Spawn 规则

借鉴 Claude Code 的经验，同一个专项智能体是否继续使用已有上下文，必须有明确规则。

第一版可以不实现真正的运行中续聊，但必须在任务记录里保留 `parent_advisor_task_id`，为后续 Continue 预留结构。

13. 统一输入：证据包

多智能体不直接扫描完整实验目录。主控流程先生成证据包，再交给协调器裁剪。

证据包至少包含：

代码块

```
1  {"session_id": "agent-20260806-001", "experiment_dir": "experiments/2026-08-06/exp1", "goal": "找到高可信降分边界候选", "snapshot_ids": {"memory": "...", "policy": "...", "prompt": "...", "operator": "..."}, "summary": {}, "observations": [], "tool_events": [], "memory_summary": {}, "evidence_refs": []}
```

要求：

1. 只放摘要和证据引用，不放完整模型响应。
2. 不放 API key、Base URL、Authorization 和完整环境变量。
3. 每个证据引用必须能定位到正式产物。
4. `snapshot_ids` 必须进入输入 hash，防止不同快照下的建议被混用。

14. 统一输出：建议结构

每个专项智能体输出同一种结构，方便汇总和校验。

代码块

```
1  {"advisor_id": "router_diagnosis","status": "completed","summary": "路由器在某类  
   题型上可能召回了不适用算子","findings": [{"type": "router_risk","severity":  
   "medium","claim": "016 在本次同类样本中多次 not_applicable","evidence_refs":  
   ["round_1/branch_results.jsonl#branch-001"],"recommended_action":  
   "needs_human_review"}],"forbidden_actions_requested": []}
```

输出限制：`summary` 不超过 800 字，`findings` 不超过 20 条，单条 `claim` 必须有证据引用。
超过限制的输出由汇总器截断，并标记为 `truncated_by_merger`。

15. 建议合成规则

协调器必须理解建议后再合成，不能把多个智能体输出原样拼接进报告。

合成顺序：

1. 先执行策略校验，越权建议直接拒绝。
2. 再检查证据引用，没有证据的建议降级为 `needs_human_review`。
3. 再检查输入 hash，证据包不一致的建议不能合并。
4. 再按风险优先级处理，高风险优先进入人工复核。
5. 多个建议冲突时，不自动选择一方，写入冲突清单。
6. 只有不冲突、证据充分、状态允许的建议才能进入 proposal。

例子：

代码块

- ```
1 策略归纳智能体建议 012 进入 shadow。
2 冲突审查智能体指出 012 在同类题中有高比例 validation_failed。
3 合成结果不能写 shadow，只能写 needs_human_review，并说明冲突原因。
```

## 16. 失败降级

多智能体失败不能影响主流程的基本产物。

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |

降级后的报告必须写清楚“缺了哪类分析”，不能把缺失分析包装成正常结论。

## 17. 成本与缓存优化

Fork Subagent 的核心价值是复用稳定上下文，降低成本和延迟。本项目暂不需要照搬 Fork，但应提前按缓存友好的方式设计输入。

可执行做法：

1. 专项智能体的公共提示词前缀保持稳定，不在前缀中拼接随机时间、随机路径或大段日志。
2. 工具列表按固定顺序生成，避免相同任务因为顺序不同导致缓存失效。
3. 大型日志只通过 `evidence_refs` 引用，不直接放进提示词。
4. 为每次调用生成 `context_cache_key`，计算内容至少包括公共提示词版本、AdvisorSpec 版本、工具白名单版本、快照 ID、`evidence_pack_slice_hash` 和模型路由策略版本。
5. 如果底层模型或调用层暂不支持 prompt 缓存，仍保留 `context_cache_key`，用于后续去重和复盘。

只有两类任务适合后续做 Fork 类优化：报告改写、同一证据包上的追加分析。专项诊断、冲突审查和验证类任务仍应使用独立上下文，避免前一个智能体的判断污染后一个智能体。

## 18. 第一类优化：实验复盘专项智能体

这是最先实施的一类多智能体。它只在实验结束后运行，不影响当前实验的正式结果。

### 18.1 路由诊断智能体

职责：判断失败是否来自算子选择不合适。

输入：样本画像、路由器输出、算子候选、运行期策略、最终分支结果。

输出：召回后多次 `not_applicable` 的算子、可能漏召回的算子、路由器过度依赖的画像字段、需要人工关注的适用条件。

禁止：直接新增、删除或重排正式算子候选。

## 18.2 算子生成诊断智能体

职责：判断问题生成失败是否来自算子本身。

输入：候选问题、父题、算子 ID、验证结果、效果分析。

输出：没有命中目标推理机制的算子、容易产生不可回答或题面泄漏的算子、提示词问题和样本不匹配问题的区分。

禁止：直接修改算子提示词。

## 18.3 校验诊断智能体

职责：判断失败是否来自可回答性、事实性或复杂度校验。

输入：校验结果、无效生成记录、候选问题、参考材料。

输出：无效生成主因、校验过严或过松的迹象、需要人工复核的候选。

禁止：把未通过校验的候选改成通过。

## 18.4 评分稳定性智能体

职责：判断评分和评分规则是否稳定。

输入：作答结果、评分规则、评分结果、重复评分记录、效果分析。

输出：评分分歧、`score_decreased` 是否可靠、`score_increased` 是否来自评分波动、是否需要人工复评。

禁止：直接改写评分或评分规则。

## 18.5 搜索成本智能体

职责：判断搜索策略和预算是否合理。

输入：搜索状态、分支数、候选数、模型调用数、耗时、终止原因。

输出：高成本低收益分支、过早停止或过度探索、下次实验预算收缩或扩大建议。

禁止：直接改变当前实验的预算消耗记录。

# 19. 第二类优化：记忆编译专项智能体

这类智能体只用于离线记忆编译。它们不能在当前实验中热更新记忆。

## 19.1 事实抽取智能体

职责：从局部实验记忆中抽取可归档事实。

输入：`operator_memory_bank.jsonl`、`failure_memory_bank.jsonl`、`invalid_generation_cases.jsonl`、效果分析和分支结果。

输出：跨实验原始账本候选。

要求：每条事实必须保留来源实验、样本、轮次、分支、算子和评分证据。

## 19.2 分类映射智能体

职责：把样本画像映射为稳定检索键。

输入：样本画像、虚高诊断、题型结构、场景字段。

输出：`scene_family`、`question_form`、`reasoning_mechanism`、`overscore_pattern`。

要求：低置信映射只能进入 `needs_human_review`，不能进入 `active`。

## 19.3 策略归纳智能体

职责：把多条实验事实归纳成策略卡。

输入：跨实验事实、分类键、算子表现统计。

输出：策略卡草案，包含适用条件、排除条件、推荐算子、备选算子、规避算子、证据强度和负收益风险。

## 19.4 冲突审查智能体

职责：检查策略卡是否和失败事实冲突。

输入：策略卡草案、失败记忆、无效生成记录、评分稳定性报告。

输出：冲突清单和风险等级。

## 19.5 发布预审智能体

职责：判断策略卡能进入哪个状态。

输入：策略卡、冲突审查结果、Replay/Holdout 结果、人工复核记录。

输出：`proposed`、`shadow`、`needs_human_review` 或 `rejected_insufficient_evidence`。

禁止：直接写 `active`。`active` 只能由发布门禁写入。

## 20. 第三类优化：计划候选专项智能体

这类智能体只生成候选计划，不直接执行计划。主控智能体只能从通过校验的候选中选择一个执行。

### 20.1 保守计划智能体

职责：生成低风险计划。

适用场景：样本风险高、预算少、评分不稳定、历史失败多。

输出特点：少分支、少候选、优先使用高置信算子，必要时请求人工复核。

## 20.2 探索计划智能体

职责：生成多算子横向探索计划。

适用场景：路由器置信度不足，多个算子都可能有效，样本探索价值高。

输出特点：多个算子并行生成候选，但必须限制分支数和候选数。

## 20.3 纵向叠加计划智能体

职责：判断是否适合两层算子叠加。

适用场景：第一层算子已经形成部分边界，但弱模型仍能答对，需要第二层推理压力。

输出特点：明确父题可回答、第一层没有负收益、第二层目标机制清晰。

## 20.4 恢复计划智能体

职责：在失败或中断后生成恢复方案。

适用场景：工具失败、manifest 损坏、`score_increased`、预算耗尽或部分产物缺失。

输出特点：明确从哪个恢复点恢复，哪些步骤不能重跑，哪些快照不能变化。

# 21. 第四类优化：人工复核预审专项智能体

这类智能体服务人工复核人员，目标是减少人工筛材料的时间。

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

复核汇总智能体不能把预审结果写成最终确认，只能说明“为什么需要人工看”。

# 22. 实施步骤

## 22.1 第一步：注册表、证据包和事件

实现内容：

1. 新增 `advisor_registry.py`、`advisor_state.py`、`advisor_events.py`。
2. 新增 `model_router.py`，根据 `model_tier` 选择具体模型，不允许专项智能体直接硬编码模型名称。
3. 新增 `evidence_pack.py`，从现有观察器和事件日志生成 `evidence_pack.json`。

4. 新增 `advisor_spec.schema.json`、`advisor_run_record.schema.json`、`advisor_advice.schema.json`。
5. 在 `advisor_spec.schema.json` 中加入 `model_tier`、`fallback_model_tier`、`requires_json_output`、`requires_evidence_refs`。
6. 给每次专项智能体运行写入 `AdvisorRunRecord`。

验收标准：没有注册表记录的智能体不能运行；没有 `model_tier` 的智能体不能运行；证据包不包含密钥、完整环境变量和完整模型响应；事件日志可追溯每个专项智能体的输入和输出 hash。

## 22.2 第二步：只读实验复盘多智能体

实现内容：

1. 新增 5 个实验复盘专项智能体。
2. 新增建议汇总器 `advice_merge.py`。
3. 在 `agent_report.md` 中增加“多智能体复盘建议”章节。

验收标准：专项智能体只读证据包；任意一个专项智能体失败不影响主报告；越权建议会被拒绝并写入报告。

## 22.3 第三步：并行调度和失败降级

实现内容：

1. 新增 `advisor_dispatcher.py` 和 `advisor_executor.py`。
2. 支持并发限制、超时、重试和跳过。
3. 根据触发规则只运行必要智能体。
4. 调度器调用 `model_router.py`，把 `model_tier` 转换为具体模型并记录到 `AdvisorRunRecord`。

验收标准：并行任务能同时启动；超时任务被标记为 `timeout`；证据缺失时相关智能体被 `skipped`；模型选择会被记录且可复盘；主报告仍完成。

## 22.4 第四步：记忆编译多智能体

实现内容：

1. 新增事实抽取、分类映射、策略归纳、冲突审查、发布预审 5 类专项智能体。
2. 输出策略卡草案和冲突清单。
3. 状态只允许 `proposed`、`shadow`、`needs_human_review`、`rejected_insufficient_evidence`。

验收标准：没有证据引用的策略卡不能通过；单次实验事实不能直接进入 `active`；冲突策略必须进入人工复核。

## 22.5 第五步：计划候选和人工复核预审

实现内容：

1. 新增保守计划、探索计划、纵向叠加计划、恢复计划 4 类专项智能体。
2. 新增边界质量、可回答性、泄漏风险、机制命中、复核汇总 5 类专项智能体。
3. 所有候选计划先进入计划校验器。

验收标准：未通过校验的计划不能执行；验证类智能体必须新建上下文；人工复核预审不能把候选写成“已确认有效边界”。

## 22.6 第六步：缓存友好和 Continue 预留

实现内容：

1. 为每个专项智能体调用写入 `context_cache_key`。
2. 固定公共提示词前缀和工具列表顺序。
3. 为每个 `evidence_pack_slice` 写入 hash，并记录在 `AdvisorRunRecord` 中。
4. 在任务记录中支持 `parent_advisor_task_id`。
5. 将专项智能体输入拆为 `advisor_static_prefix`、`advisor_spec_context`、`evidence_pack_slice` 和 `advisor_dynamic_instruction`。

验收标准：同一证据切片、同一 `AdvisorSpec`、同一工具白名单和同一提示词版本生成相同 `context_cache_key`；不同快照生成不同 key；子智能体不能读取完整父上下文；验证任务不会 Continue 旧上下文。

## 23. 建议文件结构

后续实现时可以新增以下模块：

代码块

```
1 agent_runtime/multi_agent/
2 __init__.py
3 evidence_pack.py
4 model_router.py
5 advisor_registry.py
6 advisor_context.py
7 advisor_state.py
8 advisor_events.py
9 advisor_dispatcher.py
10 advisor_executor.py
11 advice_schema.py
12 advice_merge.py
13 review_advisors.py
14 memory_advisors.py
```

```
15 planning_advisors.py
16 human_review_advisors.py
17 report_section.py
18
19 schemas/
20 advisor_spec.schema.json
21 advisor_run_record.schema.json
22 advisor_advice.schema.json
23
24 tests/
25 test_multi_agent_registry.py
26 test_multi_agent_model_router.py
27 test_multi_agent_context_isolation.py
28 test_multi_agent_policy.py
29 test_multi_agent_dispatcher.py
30 test_multi_agent_advice_merge.py
31 test_multi_agent_report_section.py
```

模块职责：

1. `evidence_pack.py` 负责生成统一输入。
2. `model_router.py` 负责把模型能力层级转换为具体模型。
3. `advisor_registry.py` 负责登记专项智能体、白名单工具和模型能力层级。
4. `advisor_context.py` 负责裁剪输入、屏蔽密钥、生成输入 hash。
5. `advisor_dispatcher.py` 负责根据触发规则选择智能体。
6. `advisor_executor.py` 负责并发、超时和重试。
7. `advice_merge.py` 负责合成、拒绝越权建议和生成冲突清单。
8. `report_section.py` 负责把多智能体建议写入报告。

## 24. 必测场景

必须覆盖以下测试：

1. 专项智能体请求修改正式提示词，结果必须被拒绝。
2. 专项智能体请求发布激活态全局记忆，结果必须被拒绝。
3. 专项智能体请求启动其它专项智能体，结果必须被拒绝。
4. 专项智能体没有证据引用，建议必须降级为 `needs_human_review`。
5. 证据包中出现 API key、Base URL 或 Authorization，生成器必须脱敏或拒绝输出。
6. 一个专项智能体失败，其它专项智能体和主报告仍能完成。
7. 一个专项智能体超时，状态必须是 `timeout`，不能写成 `completed`。

8. 两个专项智能体给出冲突建议，汇总结果必须写冲突清单。
9. 计划候选未通过计划校验器，不能进入执行器。
10. 验证类智能体必须使用新上下文，不能复用生成类智能体上下文。
11. 同一 `evidence_pack_slice`、同一 `AdvisorSpec`、同一提示词版本、同一工具白名单必须生成相同 `context_cache_key`。
12. 不同 `snapshot_ids` 的建议不能被合并到同一次 `proposal`。
13. 注册表缺少 `model_tier` 时，专项智能体不能运行。
14. 搜索和摘录类智能体使用 `extract_low_cost` 时，输出中不能包含直接执行建议。
15. 策略归纳、冲突审查、评分稳定性和建议合成不能路由到 `extract_low_cost`。
16. 模型路由结果必须写入 `AdvisorRunRecord`，包括 `model_tier`、具体模型和 fallback 是否发生。
17. 专项智能体输入中出现完整父上下文、完整实验目录或完整模型响应时，必须被上下文隔离测试拒绝。

## 25. 最终验收标准

多智能体优化完成后，应满足：

1. 主控智能体仍是唯一执行决策入口。
2. 协调器只分发、收集和合成建议，不直接执行实验工具。
3. 专项智能体只读证据包，不能直接写正式实验结果。
4. 所有专项智能体必须来自注册表，并受工具白名单限制。
5. 所有专项智能体必须声明 `model_tier`，具体模型由 `model_router.py` 统一选择。
6. 搜索、摘录、归因、策略、合成使用不同模型层级，不能用低成本模型绕过高风险判断。
7. 每次专项智能体运行都有任务记录、事件记录、输入 hash、输出 hash 和模型路由记录。
8. 所有建议都有统一结构和证据引用。
9. 专项智能体上下文按 `advisor_static_prefix`、`advisor_spec_context`、`evidence_pack_slice` 和 `advisor_dynamic_instruction` 分层，且只能读取裁剪后的证据切片。
10. 越权建议会被拒绝并进入报告。
11. 冲突建议不会被自动合并成执行计划。
12. 记忆、提示词、策略和算子快照不会被专项智能体热更新。
13. 人工复核人员可以通过报告看清每个建议的来源、风险和处理状态。

## 26. 暂不实施内容

以下内容不进入本方案：

1. 多个自治智能体同时执行真实工具。
2. 多个智能体同时写同一个实验目录。
3. 让专项智能体自动修改提示词、路由器、评分规则或算子文件。
4. 让专项智能体自动发布激活态全局记忆。
5. 让投票结果覆盖真实评分。
6. 引入通用多智能体框架替换现有流水线。
7. 第一版实现完整消息队列服务。
8. 第一版实现 Fork Subagent。

这些内容风险高，且会破坏当前项目最重要的可恢复、可审计和可复现边界。